

ปฏิบัติการที่ 1: การทบทวนความรู้ สถาปัตยกรรมแยกเลเยอร์ และการตั้งค่าเริ่มต้นโครงสร้าง Compass App

กลุ่มเป้าหมาย: นักศึกษาชั้นปีที่ 3-4	เวลาเรียนสัมมนา: 3 ชั่วโมง (ปฏิบัติหลัก)
ซอฟต์แวร์จำลองหลัก: Compass App (Google Sample)	ผู้ประสานงานหลัก: คณาจารย์ผู้สอนวิชา Flutter

1. วัตถุประสงค์การเรียนรู้ประจำปฏิบัติการ (Learning Objectives)

ในปฏิบัติการครั้งนี้ นักศึกษาจะเริ่มต้นทบทวนความรู้จากครั้งแรก และปรับเปลี่ยนกระบวนการคิดไปสู่การจัดเรียงสถาปัตยกรรมขนาดใหญ่สำหรับโครงสร้าง **Compass Application** ซึ่งเป็นแอปพลิเคชันจองทริปการท่องเที่ยวระดับโปรดัคชันของทีมผู้พัฒนา Flutter โดยมีวัตถุประสงค์เฉพาะเจาะจงดังนี้:

- ทบทวนความแตกต่างและสิทธิการทำงาน:** ระหว่าง StatelessWidget และ StatefulWidget รวมถึงกฎการระบายหน้าจ **UI is a function of (immutable) state**
- เข้าใจกระบวนการ Separation of Concerns (SoC):** การแบ่งความรับผิดชอบของโค้ดโปรแกรมออกเป็นเลเยอร์ UI, Domain, และ Data อย่างชัดเจน
- เข้าใจหลักการไหลข้อมูลทางเดียว (Unidirectional Data Flow):** และหลักการแหล่งความจริงเดียว **Single Source of Truth (SSOT)**
- ปฏิบัติงานติดตั้งคลังโปรแกรม (Dependencies Setup):** จัดเตรียม dio, provider, และ flutter_secure_storage เพื่อพร้อมใช้งาน
- สร้างโครงสร้างสถาปัตยกรรมและคลาสโมเดลโดเมนเริ่มต้น:** เพื่อสร้างโครงสร้างโมเดล BookingSummary ที่สมบูรณ์แบบ

2. ทฤษฎีสถาปัตยกรรมแอปพลิเคชันและการสื่อสารระหว่างเลเยอร์

เพื่อป้องกันปัญหาโค้ดพันกันรองรับเมื่อระบบขยายขนาด (Scalability) หรือเพิ่มความยากลำบากในการเขียนชุดควบคุมทดสอบ (Testability) สถาปัตยกรรมของแอปพลิเคชัน Compass จะกำหนดหน้าที่และความรับผิดชอบของส่วนประกอบย่อยต่างๆ โดยควบคุมผ่านกฎการเชื่อมโยงและสิทธิการติดต่อสื่อสารกัน (Rules of Engagement) อย่างเข้มงวด ดังรายละเอียดในตารางสรุปด้านล่างนี้:

ส่วนประกอบ (Component)	หน้าที่หลัก (Primary Responsibility)	กฎการเชื่อมโยงสื่อสาร (Rules of Engagement)
View (UI)	แสดงผลหน้าจอแก่ผู้ใช้งานและตรวจจับ Action สิ่งการ (เช่น การสไลด์นิ้วหรือคลิกปุ่ม) หน้าจอจะเป็นคลาสสวิตช์แบบไร้สไลด์จกซับซ้อน	รู้จักเฉพาะ ViewModel ของตัวเองตัวเดียวเท่านั้น ผ่านพารามิเตอร์ที่ส่งเข้ามาในขั้นตอนการจัดสร้างหน้าจอ
ViewModel (UI)	ควบคุม จัดการ และจัดเก็บข้อมูลสถานะชั่วคราวของหน้าจอ เพื่อพร้อมแสดงผล และเปิดฟังก์ชันทำงานแบบ asynchronous (Commands)	รู้จักคลังข้อมูล (Repositories) ตั้งแต่หนึ่งตัวขึ้นไป แต่ตัวมันจะไม่รับรู้เลยว่ามีส่วนแสดงผลตัวใดคอยสังเกตการณ์อยู่

ส่วนประกอบ (Component)	หน้าที่หลัก (Primary Responsibility)	กฎการเชื่อมโยงสื่อสาร (Rules of Engagement)
Repository (Data)	ควบคุม ดูแล และเปลี่ยนแปลงข้อมูลระดับโมเดลทั้งหมด ทำหน้าที่เป็นแหล่งความจริงเดียว (SSOT) ของประเภทข้อมูลนั้นๆ มีระบบแคชและอัปเดตข้อมูล	รู้จัก Services ในการดึงข้อมูลดิบ ถูกเรียกใช้งานต่อโดย ViewModels หลายๆ ตัวในระบบ โดยที่ตัวมันไม่รู้ตัวตนของ ViewModels เลย
Service (Data)	คลาสประมวลผลไร้สถานะ (Stateless) มีหน้าที่เดียวคือห่อหุ้ม API ภายนอกหรือฐานข้อมูลและส่งค่าดิบกลับขึ้นมาในรูปแบบโมเดล API (JSON)	ทำงานรับใช้ Repositories เท่านั้น โดยไม่มีวันรับรู้ความมีอยู่ของส่วนเชื่อมต่อแสดงผล UI หรือส่วนประกอบสถาปัตยกรรมเลเยอร์อื่นเลย

แนวคิดหัวใจสำคัญ: UI is a function of State (UI = f(State))

หน้าจอชีวิตในเลเยอร์ UI จะเป็นเพียงโครงสร้างรับสัญญาณภาพ (Declarative) ที่คอยระบายผลการแสดงขึ้นมาบนเครื่องผู้ใช้งานตามสถานะ (State) ของข้อมูลในปัจจุบันที่ ViewModel ส่งมอบให้เท่านั้น โดยลอจิกการคำนวณและการเปลี่ยนแปลงฐานข้อมูลใดๆ จะถูกซ่อนไว้อยู่เบื้องหลังเลเยอร์ข้อมูลอย่างปลอดภัย

3. ขั้นตอนปฏิบัติงานที่ 1: การตรวจสอบระบบและการจัดตั้งโปรเจกต์ใหม่

เพื่อเป็นการเตรียมพร้อมสภาพแวดล้อมในการพัฒนาโปรแกรมระบบสัมมนาวิชา ให้ทำตามคำสั่งประมวลผลและการจัดโครงสร้างในส่วนเครื่องมือพัฒนาของนักศึกษาอย่างละเอียดที่ละขั้นตอนดังนี้:

ขั้นตอนที่ 1.1: ตรวจสอบความพร้อมของระบบ SDK

เปิดโปรแกรม Command Prompt หรือ Terminal ขึ้นมาบนเครื่องคอมพิวเตอร์ แล้วพิมพ์คำสั่งด้านล่างเพื่อตรวจสอบความสมบูรณ์เรียบร้อยของ Flutter SDK และโปรแกรมจำลอง Emulator สำหรับทดสอบรันหน้าจอ:

```
$ flutter doctor
```

ขั้นตอนที่ 1.2: จัดตั้งโปรเจกต์แอปพลิเคชัน Compass ใหม่

ให้นักศึกษาพิมพ์คำสั่งประมวลผลเพื่อสร้างโฟลเดอร์โปรเจกต์ Flutter ตัวใหม่ของพวกเรา และเปลี่ยนหน้าต่างการทำงานเข้าสู่ไดรেকทอรีโปรเจกต์ เพื่อเตรียมเริ่มวางโครงสร้างไฟล์ระบบข้อมูลหลัก:

```
# สร้างโปรเจกต์ Flutter ตัวใหม่ในโครงสร้างสถาปัตยกรรม Compass
$ flutter create compass_booking_app

# เปลี่ยนหน้าต่างควบคุม (Terminal) เข้าสู่โฟลเดอร์โปรเจกต์
$ cd compass_booking_app
```

ขั้นตอนที่ 1.3: นำเข้าแพ็คเกจคลังไลบรารีเพื่อสนับสนุนการทำงาน

พิมพ์คำสั่งเพิ่มไลบรารีที่จำเป็นต่อการพัฒนาแอปพลิเคชันเพื่อเชื่อมต่อ REST API ระบบควบคุมสถานะส่วนกลาง และการจัดสรรเก็บความลับ JWT Token อย่างปลอดภัยลงบนอุปกรณ์จำลอง:

```
$ flutter pub add dio provider flutter_secure_storage
```

รายละเอียดไลบรารีประจำโปรเจกต์ที่นำมา:

- **dio (v5.x+):** คลังไลบรารีสำหรับการเรียกใช้ REST API และสร้าง HTTP Client สนับสนุน Interceptors ดักจับข้อมูลอย่างรวดเร็วและปลอดภัย
- **provider:** เครื่องมือและกลไกในการจัดการพึ่งพาและความสัมพันธ์ของอินสแตนซ์ในระบบสถาปัตยกรรม (Dependency Injection)
- **flutter_secure_storage:** ไลบรารีสำหรับการเก็บรักษา Access Token และ Refresh Token ของ JWT บนพื้นที่ความปลอดภัยระดับระบบปฏิบัติการตัวเครื่อง

4. ขั้นตอนปฏิบัติงานที่ 2: การจัดโครงสร้างโฟลเดอร์ตามสถาปัตยกรรม

เพื่อรักษาหลักระเบียบการจัดวางโค้ดสากลระดับโปรดัคชัน of Compass App ให้นักศึกษาจัดกลุ่มข้อมูลส่วนกลาง (Data, Domain) ไว้ตามประเภท (By Type) และจัดกลุ่มข้อมูลการแสดงผล (UI) ไว้ตามแต่ละส่วนฟีเจอร์การทำงานของเครื่องมือแสดงผล (By Feature) โดยใช้คำสั่งจัดโครงสร้างดังต่อไปนี้:

ขั้นตอนที่ 2.1: การใช้คำสั่งสร้างโฟลเดอร์ปฏิบัติงาน

พิมพ์คำสั่งสร้างโครงสร้างโฟลเดอร์ความรับผิดชอบย่อยภายใต้ไดเรกทอรี lib/ ของโปรเจกต์ตามสถาปัตยกรรม MVVM ให้เสร็จสมบูรณ์:

```
# สำหรับระบบปฏิบัติการ Unix / macOS Terminal:
$ mkdir -p lib/data/models lib/data/repositories lib/data/services lib/domain/models lib/ui/core
lib/ui/home/view_models lib/ui/home/widgets

# สำหรับระบบปฏิบัติการ Windows PowerShell:
$ mkdir lib\data\models, lib\data\repositories, lib\data\services, lib\domain\models, lib\ui\core,
lib\ui\home\view_models, lib\ui\home\widgets
```

5. ขั้นตอนปฏิบัติงานที่ 3: การตั้งค่าระบบผู้ช่วยพัฒนา AI และ MCP Server

เพื่อขยายขอบเขตความรู้ความสามารถของนักศึกษาวิชา Flutter ยุคใหม่ เราจะผูกเครื่องมือเขียนโค้ดสนับสนุน AI (เช่น Claude Code, Cursor, หรือ VS Code) เข้ากับ **Model Context Protocol (MCP)** เพื่อช่วยตรวจสอบสเปกโครงสร้างแอปและแก้ไขตรรกะแบบเรียลไทม์ระหว่างจัดสร้างใบงาน ดังนี้:

ขั้นตอนที่ 3.1: ติดตั้งปลั๊กอินและสิทธิ์ความรู้สำหรับผู้ช่วย AI

พิมพ์คำสั่งเพิ่มความรู้ทางด้าน Dart & Flutter ให้กับเครื่องผู้ช่วย AI ในหน้าต่าง Terminal (สำหรับ Claude Code เป็นหลัก):

```
# เพิ่มสเปกคำสั่งและ Blueprint ความเป็นเลิศจาก Flutter Lang
$ claude plugin marketplace add flutter/agent-plugins

# ติดตั้งแพ็คเกจปลั๊กอินเพื่อสนับสนุนการขับเคลื่อนตัวค้นแรง UI
$ claude plugin install dart-flutter@dart-flutter
```

6. ขั้นตอนปฏิบัติงานที่ 4: การจัดทำโครงร่างคลาสข้อมูลระดับโดเมน (Domain Model)

สเปกขั้นตอนปฏิบัติสุดท้ายในสัปดาห์ที่ 1 นี้ คือการร่วมเขียนโครงร่างคลาสสเกเลตัน (Skeleton Code) ของประเภทข้อมูลกริบัติทางท่องเที่ยวชื่อคลาส BookingSummary ภายใต้ไดเรกทอรีส่วนกลาง lib/domain/models/booking.dart เพื่อทำหน้าที่เป็นสถาปัตยกรรมโมเดลชั้นข้อมูลระดับธุรกิจเดียวในการแสดงผลปลายทาง:

ขั้นตอนที่ 4.1: การเขียนโค้ด Dart ในไฟล์คลาส BookingSummary

ให้นักศึกษาร่างไฟล์ใหม่ชื่อ booking.dart ขึ้นมาภายใต้โฟลเดอร์ lib/domain/models/ และพิมพ์ชุดโค้ดโปรแกรมภาษา Dart ต่อไปนี้ลงในไฟล์ให้เรียบร้อย:

```
// ไฟล์คลาสโมเดลข้อมูลส่วนกลางของระบบ: lib/domain/models/booking.dart

class BookingSummary {
  final int id; // หมายเลขประจำตัวของทริปการท่องเที่ยว
  final String destinationName; // ชื่อสถานที่ท่องเที่ยวหรือเมืองจุดหมายปลายทาง
  final DateTime startDate; // วันและเวลาที่เริ่มออกเดินทาง
  final DateTime endDate; // วันและเวลาสิ้นสุดทริปการท่องเที่ยว
  final double price; // ราคาค่าบริการจองของทริปการท่องเที่ยวนี้

  // คอนสตรัคเตอร์สำหรับสร้างอินสแตนซ์ของคลาสทริปเดินทาง
  BookingSummary({
    required this.id,
    required this.destinationName,
    required this.startDate,
    required this.endDate,
    required this.price,
  });
}
```

สรุปการเรียนรู้และความพร้อมปฏิบัติการครั้งถัดไป:

ยินดีด้วย! นักศึกษาได้ผ่านปฏิบัติการสัปดาห์ที่ 1 เรียบร้อยแล้ว ขณะนี้คุณเข้าใจโครงข่ายสถาปัตยกรรม **MVVM** ที่ทีมงาน Google แนะนำ และสามารถวางโครงสร้างคลาสโดเมน BookingSummary ในไฟล์ได้เสร็จสมบูรณ์ ในสัปดาห์ถัดไป (สัปดาห์ที่ 2) เราจะร่วมกันพัฒนาเครื่องฐานข้อมูลระบบหลังบ้านดึงข้อมูลดิบอย่างแท้จริงโดยใช้ **Django REST Framework (DRF)** และระบบความปลอดภัยอีกอัน **JSON Web Token (JWT)** ขอให้นักศึกษาทุกคนศึกษาแนวทางพื้นฐานของ Django REST และเครื่องมือเขียนคำสั่งมาล่วงหน้าเพื่อเตรียมพร้อมสำหรับการทำงานจริง!